Assignment

An assignment for class **MET CS 566** at Boston University, instructed by **Ming Zhang**. Student Zhang, Jin Yu submitted on **October 20, 2022**.

Question 1

Ordering by asymptotic growth rates. Rank the following functions by order of growth. This means finding an arrangement $g_1, g_2, ..., g_9$ of the functions that satisfy $g_1 = \Omega(g_2), g_2 = \Omega(g_3), ..., g_7 = \Omega(g_8)$:

1. $g_1 = \log(\log(n))$
2. $g_2 = 2^{\log(n)}$
3. $g_3 = 2\sqrt{2 \times \log(n)}$
4. $g_4 = n^{8.2} + \log(n)$
5. $g_5 = n^{8.2} + n!$
6. $g_6 = n^{2024} + n^{512} + n \times \log(n)$
7. $g_7 = e^n + e^{ln(n)}$
8. $g_8 = \sqrt{\log(n)}$

Work in stepwise format. Find the fastest growing function first, and then the next fast one.

Solution

The final result. Please see stepwise solution below:

1. $g_1 = \log(\log(n))$
2. $g_8 = \sqrt{\log(n)}$
3. $g_3 = 2^{\sqrt{2} \times \sqrt{\log(n)}}$
4. $g_2 = n$
5. $g_4 = n^{8.2}$
6. $g_6 = n^{2024}$
7. $g_7 = e^n$
8. $g_5 = n!$

Refine some functions to get rid of lower-order terms in order to compare them precisely:

1. $g_1 = \log(\log(n))$
2. $g_2 = 2^{\log(n)} = n$
3. $g_3 = 2\sqrt{2 \times \log(n)} = 2^{\sqrt{2} \times \sqrt{\log(n)}}$
4. $g_4 = n^{8.2} + \log(n) = n^{8.2}$
5. $g_5 = n^{8.2} + n! = n!$
6. $g_6 = n^{2024} + n^{512} + n \times \log(n) = n^{2024}$
7. $g_7 = e^n + e^{ln(n)} = e^n + e^{\log_e^n} = e^n + n = e^n$
8. $g_8 = \sqrt{\log(n)}$

First of all, we compare g_1 and g_8 since they have a similar form. $g_1 = \log(n)$ and $g_8 = \sqrt{\log(n)}$. Assume $\log(n)$ is v, then $g_1 = \log(v) < g_8 = \sqrt{v}$. Now we have:

1. $g_1 = \log(\log(n))$
2. $g_8 = \sqrt{\log(n)}$

Then we put $g_2 = n$ in the list. g_2 is obviously slower than g_1 and g_8 according to **Order of Growth of Functions** mentioned in lectures:

1. $g_1 = \log(\log(n))$
2. $g_8 = \sqrt{\log(n)}$
3. $g_2 = n$

$g_4 = n^{8.2}$ and $g_6 = n^{2024}$ are quite similar to $g_2 = n$ and can be compared easily:

1. $g_1 = \log(\log(n))$
2. $g_8 = \sqrt{\log(n)}$
3. $g_2 = n$
4. $g_4 = n^{8.2}$
5. $g_6 = n^{2024}$

Use the definition of **Order of Growth of Functions** again for g_7 and g_5. Factorial rates are slower than exponential rates, and exponential rates are slower than polynomial rates:

1. $g_7 = e^n$
2. $g_5 = n!$

Put them into the list:

1. $g_1 = \log(\log(n))$
2. $g_8 = \sqrt{\log(n)}$
3. $g_2 = n$
4. $g_4 = n^{8.2}$
5. $g_6 = n^{2024}$
6. $g_7 = e^n$
7. $g_5 = n!$

It seems that there is no function has the same base with $g_3 = 2^{\sqrt{2} \times \sqrt{\log(n)}}$. We have to simplify it and find a common base in order to compare:

$$g_3 = 2^{\sqrt{2} \times \sqrt{\log(n)}} = 2^{\sqrt{\log(n)} \times \sqrt{2}} = 2^{\sqrt{\log(n)}^{\sqrt{2}}} = n^{\frac{\sqrt{2}}{2}}. \log(n) \leq g_3 \leq n$$

Therefore, the list will be:

1. $g_1 = \log(\log(n))$
2. $g_8 = \sqrt{\log(n)}$
3. $g_3 = 2^{\sqrt{2} \times \sqrt{\log(n)}}$
4. $g_2 = n$
5. $g_4 = n^{8.2}$
6. $g_6 = n^{2024}$
7. $g_7 = e^n$
8. $g_5 = n!$

Question 2

Asymptotic notations. For each pair of the following functions $f(n)$ and $g(n)$, check if:

1. $f(n) = \Theta(g(n))$
2. $f(n) = \mathcal{O}(g(n))$
3. $f(n) = \Omega(g(n))$
4. $f(n) = o(g(n))$
5. $f(n) = \omega(g(n))$

Functions $f(n)$ and $g(n)$ are:

1. $f(n) = 128^{\frac{n}{4}}\ vs.\ g(n) = 512^{\frac{n}{8}}$
2. $f(n) = n^{64} \times 2^n\ vs.\ g(n) = \log(n) \times 2^n + 4^n + n^{32}$
3. $f(n) = n^{1024}\ vs.\ g(n) = 2^{\log(n) \times \log(n)}$

You do not need to provide a formal proof. Describe your justifications in text form.

Solution

$f(n) = 128^{\frac{n}{4}}\ vs.\ g(n) = 512^{\frac{n}{8}}$

We first change the form:

$$f(n) = 128^{\frac{n}{4}} = \sqrt[2^2]{128}^{n} = \sqrt[2^2]{2^7}^{n} = 2^{\frac{7\times n}{4}} = 2^{1.75\times n}$$

$$g(n) = 512^{\frac{n}{8}} = \sqrt[2^3]{512}^{n} = \sqrt[2^3]{2^9}^{n} = 2^{\frac{9\times n}{8}} = 2^{1.125\times n}$$

$$f(n) = 2^{1.75\times n}\ vs.\ g(n) = 2^{1.125\times n}$$

- $f(n) = \Theta(g(n))$
  - **No**. Ignore lower-order terms, then $f(n) = g(n)$ is false.
- $f(n) = \mathcal{O}(g(n))$
  - **No**. Ignore lower-order terms, then $0 \leq f(n) \leq c \times g(n)$ is false.
- $f(n) = \Omega(g(n))$
  - **Yes**. Ignore lower-order terms, then $0 \leq c \times g(n) \leq f(n)$ is true.
- $f(n) = o(g(n))$
  - **No**. Ignore lower-order terms, then $0 \leq f(n) < g(n)$ is false.
- $f(n) = \omega(g(n))$
  - **Yes**. Ignore lower-order terms, then $0 \leq c \times g(n) < f(n)$ is true.

$f(n) = n^{64} \times 2^n\ vs.\ g(n) = \log(n) \times 2^n + 4^n + n^{32}$

We first change the form:

$$f(n) = 2^n$$

$$g(n) = 4^n$$

$$f(n) = 2^n\ vs.\ g(n) = 4^n$$

- $f(n) = \Theta(g(n))$
  - **No**. Ignore lower-order terms, then $f(n) = g(n)$ is false.
- $f(n) = \mathcal{O}(g(n))$
  - **Yes**. Ignore lower-order terms, then $f(n) \leq g(n)$ is true.
- $f(n) = \Omega(g(n))$
  - **No**. Ignore lower-order terms, then $f(n) \geq g(n)$ is false.
- $f(n) = o(g(n))$
  - **Yes**. Ignore lower-order terms, then $f(n) < g(n)$ is true.
- $f(n) = \omega(g(n))$
  - **No**. Ignore lower-order terms, then $f(n) > g(n)$ is false.

$f(n) = n^{1024}\ vs.\ g(n) = 2^{\log(n) \times \log(n)}$

We first change the form:

$$f(n) = n^{1024} = n^{2^{10}}$$

$$g(n) = 2^{\log(n) \times \log(n)} = (2^{\log(n)})^{\log(n)} = (n^{\log(2)})^{\log(n)} = n^{\log(n)}$$

$$f(n) = n^{2^{10}}\ vs.\ g(n) = n^{\log(n)}$$

- $f(n) = \Theta(g(n))$
  - **No**. Ignore lower-order terms, then $f(n) = g(n)$ is false.
- $f(n) = \mathcal{O}(g(n))$
  - **Yes**. Ignore lower-order terms, then $f(n) \leq g(n)$ is true.
- $f(n) = \Omega(g(n))$
  - **No**. Ignore lower-order terms, then $f(n) \geq g(n)$ is false.
- $f(n) = o(g(n))$
  - **Yes**. Ignore lower-order terms, then $f(n) < g(n)$ is true.
- $f(n) = \omega(g(n))$
  - **No**. Ignore lower-order terms, then $f(n) > g(n)$ is false.

Question 3

Big O notation. What is the running time of this code in Big O notation regarding n?

```
1. y = 0
2. j = 1
3. while (j * j <= n) do
4.     y = y + 1
5.     j = j + 1
6. end while
```

Line 4 is some constant cost.

Solution

Apparently, `y` is not really involved in developing the running time since it only takes a constant time and not a factor for any loops. $\mathcal{O}$ is derived from the fact that the while-loop would execute when $j^2 \leq n$, and `j = j + 1` after every loop:

$$\mathcal{O}(f(n)) = \sqrt{n}$$

Question 4

Big O notation. What is the running time of this code in Big O notation regarding n?

```
1. a = 0
2. k = n * n
3. while (k > 1) do
4.    for j <- 0 to n*n do
5.         a = a + j
6.    end for
7.    k = k / 2
8. end while
```

Solution

We can first assume that `a` is not a impact to the running time since `a` is not used as an index or length of execution.

The time complexity for the while-loop at line 3 is $\log(n^2)$ because we run `k = k / 2` every loop, and `k` starts at `n * n`. The time complexity for the for-loop at line 4 is n^2 since `j` only exists in the for-loop scope and starts from `0` to `n * n` every time:

$$\mathcal{O}(f(n)) = \log(n^2) \times n^2 = 2 \times \log(n) \times n^2 = \log(n) \times n^2$$

Question 5

Recursive function. What is the running time of the following python code in Big O notation regarding n?

```
def isThis(n):
    print(n)
    if (n == 1):
        return True
    if (n == 0):
        return False
    if ((n % 3) == 0):
        return isThis(n / 3)
    else:
        return False
```

Solution

Only remainder for `n % 3` equals to 0 will lead to another call stack, hence, the worst case should be n is a $3 \times m$ for any positive integer m such as 9, 27, ..., 300.

For example, when n comes to 9, it calls another call on stack with $n = 3$, then calls another one with $n = 0$. The result is `False` for $n = 9$.

$$\mathcal{O}(f(n)) = \log_3^n$$

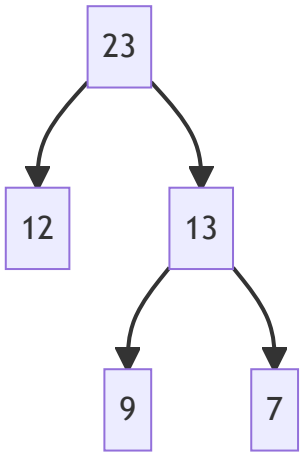
Question 6

`HEAP-EXTRACT-MAX` worst case. Describe when the worst case of `HEAP-EXTRACT-MAX` happens after you removed a max value from a heap. Provide an example.

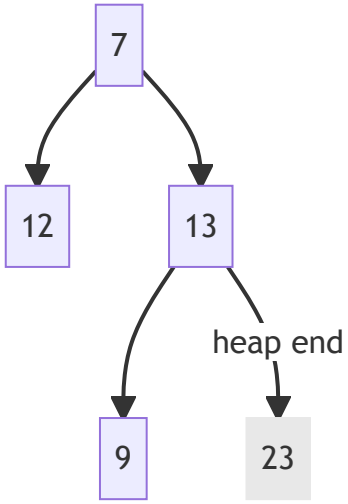
Solution

The main factor of the time complexity for `HEAP-EXTRACT-MAX` is the number of running times of `MAX-HEAPIFY`. Every time we swap the max value at the first index with the element at the last index, we have to execute `MAX-HEAPIFY(Array, 0)` to maintain the heap property for the given array. Thus, the worst cast of `HEAP-EXTRACT-MAX` happens when we have to run `MAX-HEAPIFY` from head to bottom to maintain the heap property, which is $\mathcal{O}(\log(n))$.

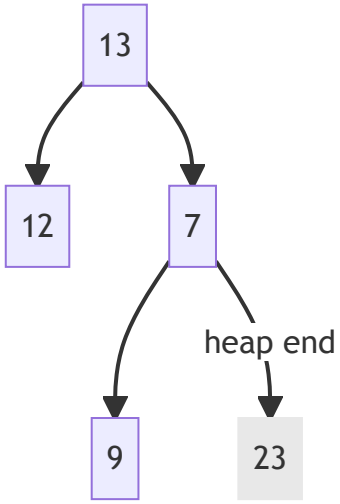
For example, we have the following heap `(23 13 12 9 7)`:



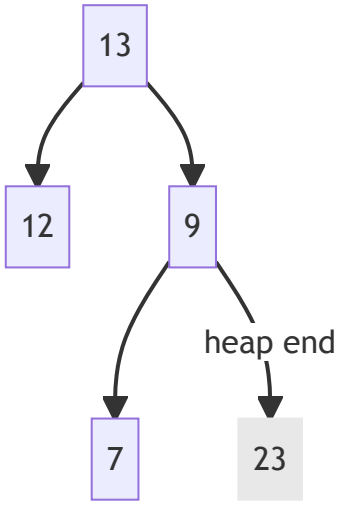
Once we execute `HEAP-EXTRACT-MAX`, we swap the max value 23 at the first index with the element at the last index 7, and then reduce the heap size by 1:



As we can see, 7 is the smallest value in the heap. When we run `MAX-HEAPIFY` we will have to run from head to bottom. 7 and 13 are exchanged:



Then 7 and 9 are changed:



Question 7

Hash function. Name two properties a good hash function should have. Describe your answer (if possible, with an example).

Solution

Minimizing the chance that similar variants hash to the same slot. For example, when we are hashing closely related symbols such as *pt* and *pts*, the result value for them through division method with their first character's ASCII code 112 may be the same, which would lead to collisions. A good hash function should distribute these 2 to different slots, or at least minimize the chance that hashing them to the same slot.

Should be very fast compute. We generally want hash functions fast, just as we want other functions to be fast. However, there are some exceptions. Applications generating passwords or encryption keys must be more secure against a brute-force attack. It is better to have slow hash functions to increase the time and space for the attacker to test each possible key.

Question 8

Double hashing. When you use double hashing in a hash table, does the number of potential collisions depend on the sequence of key insertions into the table? In other words, if you change the sequence of insertions, would the number of collisions change? Support your answer using an example.

Solution

Yes. The main reason is that we go different slot with different value. Assume there are values A and B, A occupies position `0` and B is inserted later, B may need to go to position 2 instead. If B occupies position `0` first, then A may need to go to position 3 where C will go after, which leads additional collisions.

Consider inserting the keys `{31, 11, 5, 17, 25}` into a hash table of length $m = 7$ using double Hashing. k is the input value, and i is the number of collisions:

$$h(k, i) = (h_1(k) + i \times h_2(k)) \mod m$$

$$h_1(k) = k \mod 7$$

$$h_2(k) = 1 + (k \mod 3)$$

There are 3 collisions:

1. Insert 31 at position ((31 % 7) + (0 * (1 + (31 % 3))) % 7) = 3
2. Insert 11 at position ((11 % 7) + (0 * (1 + (11 % 3))) % 7) = 4
3. Insert 5 at position ((5 % 7) + (0 * (1 + (5 % 3))) % 7) = 5
4. Insert 17 at position ((17 % 7) + (0 * (1 + (17 % 3))) % 7) = 3
5. Insert 17 at position ((17 % 7) + (1 * (1 + (17 % 3))) % 7) = 6
6. Insert 25 at position ((25 % 7) + (0 * (1 + (25 % 3))) % 7) = 4
7. Insert 25 at position ((25 % 7) + (1 * (1 + (25 % 3))) % 7) = 6
8. Insert 25 at position ((25 % 7) + (2 * (1 + (25 % 3))) % 7) = 1

Let's change the insertion sequence to `{25, 31, 17, 5, 11}`. There are only 2 collisions:

1. Insert 25 at position ((25 % 7) + (0 * (1 + (25 % 3))) % 7) = 4
2. Insert 31 at position ((31 % 7) + (0 * (1 + (31 % 3))) % 7) = 3
3. Insert 17 at position ((17 % 7) + (0 * (1 + (17 % 3))) % 7) = 3
4. Insert 17 at position ((17 % 7) + (1 * (1 + (17 % 3))) % 7) = 6
5. Insert 5 at position ((5 % 7) + (0 * (1 + (5 % 3))) % 7) = 5
6. Insert 11 at position ((11 % 7) + (0 * (1 + (11 % 3))) % 7) = 4
7. Insert 11 at position ((11 % 7) + (1 * (1 + (11 % 3))) % 7) = 0